

ArgusChain v4

Final Architecture Specification

DID-anchored identity · NFT-native assets · role tokens with per-asset access control
Separation of duty · immutable audit trail · advisory behavioral threat detection

Synthesis of ArgusChain v2 and v3, rebuilt around the problem statement's literal requirements.

Problem statement	Blockchain-Based Secure Platform for Identity, Access Control, and Digital Asset Management
PS ID / theme / category	26125 · Blockchain and Cybersecurity · Software
Organization	Bharat Electronics Limited (BEL)
Team	Blackfyre
Document status	v4 final design. Supersedes ArgusChain v2 and v3.
Date	20 September 2026
Target network	Ethereum Sepolia (demo) · Hyperledger Besu QBFT (production path)

Reading guide

Section 2 shows how every line of the problem statement is met. Sections 3–12 specify the architecture. Section 13 is the build plan with a cut line, Section 14 the demo script, Section 15 the judge Q&A. Appendix A traces every v2 and v3 feature to its v4 disposition.

Contents

1. Executive summary and the v4 decision	3
2. Problem-statement compliance matrix	4
3. System architecture	5
4. Identity layer	7
5. Access control: roles and per-asset permissions	8
6. Asset layer: NFT ownership and integrity	12
7. Storage and cryptography	13
8. Authorization pipeline	14
9. Audit trail	15
10. Behavioral threat detection (UEBA)	16
11. Smart-contract security and testing	17
12. Contract inventory and data model	19
13. Build plan: three tiers and a cut line	20
14. Demo script	21
15. Anticipated judge questions	22
16. Known risks and honest scope	23
Appendix A. Traceability: v2 and v3 features in v4	24
Appendix B. Changes needed in the SIH presentation	25

Architecture at a glance

Identity: ERC-1056 `did:ethr` + soulbound ERC-721. Roles: ERC-1155 (Admin, Manager, Auditor, User, plus Security Officer). Assets: ERC-721 with immutable SHA-256 hash, Admin-only mint, controlled transfer. Permissions: per-asset `AccessRegistry` with bits, deny, expiry, justification. Approvals: four-eyes for classified grants. Audit: native events plus Merkle-anchored decision log. Security: cryptographic authentication, contract hygiene, advisory UEBA. Encryption: per-file DEK wrapped by a Vault KEK.

1. Executive summary and the v4 decision

ArgusChain v4 is the final design. It keeps v2's answer to the problem statement (a real DID for every user, ERC-721 asset NFTs minted only by administrators, ERC-1155 role tokens for Admin, Manager, Auditor and User) and adds v3's answer to the question v2 could not answer: **who may do what to which asset, until when, and who had to approve it.**

v4 in one line

v4 = v2's PS-literal token core + v3's permission engine and separation of duty + v2's three-layer security model. v3 ideas that drifted away from the problem statement (no NFTs, no DID) or exceed hackathon scope (namehash folder tree, client-side end-to-end encryption, upgradeable proxies) are kept as documented roadmap items, not built and not claimed.

1.1 The platform in six sentences

- **Identity.** Every user gets a `did:ethr` identifier (ERC-1056) bound to a non-transferable identity NFT. Control of the DID is proven with a signature that is verified *on chain* at registration and again at every session start.
- **Roles.** The four problem-statement roles (Admin, Manager, Auditor, User) are ERC-1155 role tokens: granting a role is a mint, revoking is a burn, and peer-to-peer transfer is disabled.
- **Assets.** Every digital asset is an ERC-721 NFT carrying an immutable SHA-256 content hash. Only a caller holding the Admin role token can mint one, and it is allocated directly to a user identity. Ordinary ERC-721 transfers are disabled, so assets cannot be duplicated or reassigned outside a controlled path.
- **Access.** Roles decide who may perform platform operations. A purpose-built `AccessRegistry` decides who may do what to each individual asset, with permission bits, explicit deny, expiry, and a mandatory written justification.
- **Separation of duty.** The account that grants access cannot read the content. Auditors see everything about a file except the file. Grants on classified assets need a second, different approver.
- **Audit and security.** Every identity, role, asset, permission and transfer change is a native on-chain event. Cybersecurity is delivered as three separately defensible layers: cryptographic authentication, smart-contract hygiene, and advisory behavioral threat detection (UEBA).

1.2 What v4 takes from v2 and v3

Topic	v2	v3	v4 decision
Identity	did:ethr + soulbound NFT + challenge-response	Soulbound ID + SIWE; no DID	DID + soulbound ID + SIWE. Grants key on <code>identityId</code> ; controller is read live from ERC-1056, so key rotation keeps access.
Assets	ERC-721 with SHA-256 hash	Registry nodes; no NFT , no mint	ERC-721 assets with admin-only mint, unique-hash rule and controlled transfer. Permissions keyed by a <code>resourceId</code> that can later become a folder-tree node.
Roles	ERC-1155: Admin, Manager, Auditor, User	Designations and functional roles	The four PS roles unchanged, plus one extension role (Security Officer) needed for separation of duty.
Permissions	Role balance only (class-level)	ACE bitmasks, deny, expiry, delegation	Two tiers: role tokens gate platform operations; per-asset ACEs gate what each identity may do to each asset.
Separation of duty	None (Admin can do everything)	Full matrix + four-eyes + timelock	Adopted. Four-eyes from CONFIDENTIAL up; timelocks and break-glass in Tier 2.
Audit	Native token events	Events + hash-chained log + Merkle anchors	Both: on-chain events for state changes, Merkle-anchored log for reads and denials.
File encryption	Server-side envelope (Vault transit)	Client-side E2EE (ECIES, escrow)	Server-side envelope encryption for the build; client-side E2EE is the documented upgrade (Section 7.3).
Security model	Three layers; AI advisory	UEBA v3, invariants, peer baselines	Three layers retained; peer-group baselines and Foundry invariants added.
Upgradeability	Plain contracts	UUPS proxies on everything	Immutable contracts for the build; UUPS behind a timelock and multisig is the production path.
Scope	Small, buildable	Very large	Three explicit tiers with a cut line (Section 13).

1.3 Design principles

- **P1 Problem statement first.** Every literal requirement maps to one named mechanism (Section 2). Extras sit on top of that core and never replace it.
- **P2 Tokens where tokens fit, a registry where they do not.** Identity, roles and assets are tokens. Fine-grained, expiring, deniable per-asset permissions are not a token problem, so they live in a purpose-built registry.
- **P3 Authentication is not authorization.** A signature proves who is asking. Roles and ACEs decide what that identity may do. The two checks are separate, ordered, and separately demonstrable.
- **P4 The chain is the source of truth.** Postgres is a materialised view rebuilt from chain events and never written by application logic. SECRET and TOP_SECRET decisions read the chain directly.
- **P5 AI advises, deterministic code decides.** No model can grant access, mint or burn a token, or write a rule on chain.
- **P6 Separation of duty.** No single account can both grant access to content and read it.
- **P7 No custom cryptographic primitives.** Audited standards and libraries only: ERC-1056, ERC-721, ERC-1155, ERC-5192, EIP-4361, OpenZeppelin, AES-256-GCM, HashiCorp Vault.
- **P8 Honest scope.** Only Tier 1 and Tier 2 items are claimed as built. Tier 3 is presented as roadmap.

2. Problem-statement compliance matrix

The left column quotes the problem statement's own wording. Each row names the mechanism that satisfies it, where it is specified, and which demo step proves it live (Section 14). Tier: T1 = must build, T2 = should build.

Problem statement requires	v4 mechanism	Where	Demo	Tier
A decentralized identifier for each user, independent of central authorities	did:ethr identifier from the ERC-1056 registry (deployed unmodified). ArgusChain stores no passwords or credentials.	§4.1	1	T1
Authenticated using cryptographic proofs	Registration requires the DID owner's EIP-712 signature, verified on chain. Every session starts with a SIWE (EIP-4361) signature whose signer must equal the live ERC-1056 owner. Sensitive actions need a fresh step-up signature.	§4.2–4.4	1, 2, 9	T1
Each user identity is unique and verifiable	Soulbound ERC-721 identity NFT. Contract-enforced uniqueness: one identity per employee commitment, one per DID.	§4.3	3	T1
Digital assets are NFTs: unique, traceable, permanently recorded	ERC-721 AssetNFT . Immutable SHA-256 contentHash ; a hash can be minted only once; full history in events.	§6	4, 12	T1
NFTs are directly allocated to user identities	Mint target is a registered identity: ownerIdentity is authoritative and the ERC-721 holder is that identity's DID address.	§6.1	4	T1
Only authorized administrators create NFTs and assign them	mint() checks on chain that the caller's identity holds the Admin role token. CONFIDENTIAL and above additionally need a second approver.	§6.1, §5.7	4, 8	T1
Prevent unauthorized duplication or reassignment	Duplicate content hash reverts. Standard ERC-721 transfers always revert; only controlledTransfer() moves an asset, with a justification and required approvals.	§6.2	4	T1
RBAC with roles such as Admin, Manager, Auditor, User, defined and assigned by administrators	ERC-1155 RoleRegistry , one token id per named role. Grant = mint, revoke = burn, peer transfer disabled. Extension role: Security Officer.	§5.2	5	T1
Specific access rights per identity, enforced automatically by smart contracts	AccessRegistry : per-asset permission bits, explicit deny, expiry, justification. Effective permissions are computed by an on-chain view function.	§5.4–5.6	6, 7	T1
All identity, NFT, allocation, access-right, transfer and permission events immutably recorded	One coherent trail: native ERC-721/1155 events plus purpose-built events for identity, ACE and proposal changes; reads and denials are Merkle-anchored.	§9	13	T1 / T2

Problem statement requires	v4 mechanism	Where	Demo	Tier
Transparent, tamper-proof verification of ownership, permissions and history	Audit view links every row to a transaction hash. Files are re-verified against the on-chain hash on the server and in the browser.	\$6.3, \$9	12, 13	T1
Theme: Blockchain and Cybersecurity	Three layers: (1) cryptographic authentication, (2) smart-contract hygiene, (3) behavioral threat detection. Plus envelope encryption and separation of duty.	\$4, \$10, \$11	2, 10, 11	T1 / T2

Framing that survives questioning

Rows 1–11 are satisfied by the blockchain core alone. The behavioral (UEBA) layer answers the “Cybersecurity” half of the theme name and is presented as an addition on top of a system that already meets the problem statement without it. A challenge to the AI layer therefore never puts the core claim at risk.

3. System architecture

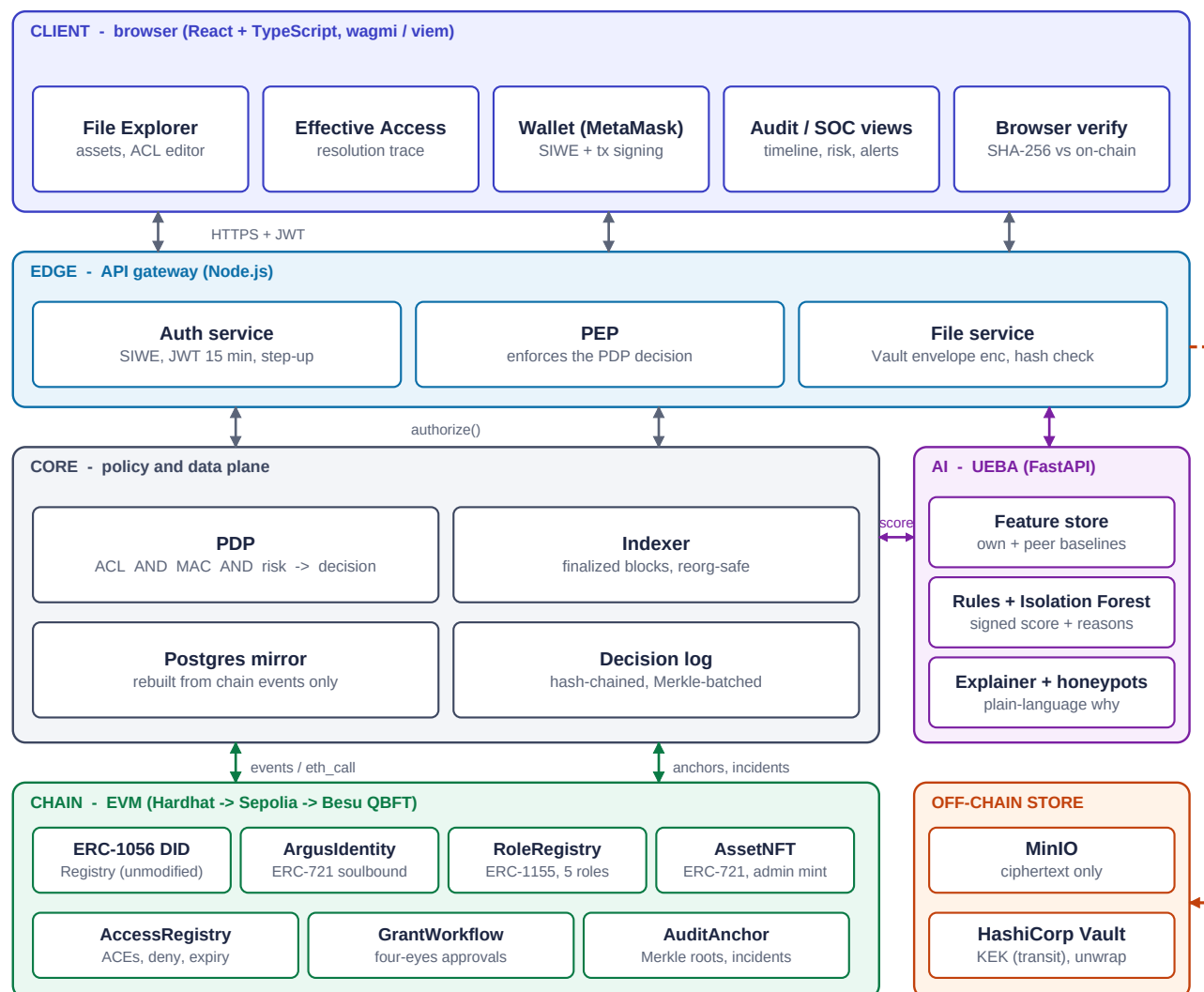


Figure 1. ArgusChain v4 layers. The dashed orange line is the private-network path between the file service, MinIO and Vault; clients never reach storage directly.

3.1 Layers and responsibilities

Layer	Components	Responsibility
Client	File explorer, ACL editor, Effective Access view, audit and SOC views, MetaMask	Presentation, wallet signing (SIWE, transactions, step-up), browser-side SHA-256 verification of downloaded files.
Edge	Auth service, Policy Enforcement Point (PEP), file service	Session issuance, calling the PDP for every request, streaming decrypted files only after an ALLOW decision.

Layer	Components	Responsibility
Core	PDP, indexer, Postgres mirror, hash-chained decision log	The deterministic policy decision (ACL and MAC and risk), a chain-derived read model, and an append-only audit trail.
AI	Feature store, rules, Isolation Forest, explainer, honeypots	Advisory risk score with plain-language reasons. Stateless and horizontally scalable. No write authority.
Chain	DID registry, identity SBT, role tokens, asset NFTs, AccessRegistry, GrantWorkflow, AuditAnchor	Source of truth for identity, roles, assets, permissions and approvals; emits the immutable audit events.
Off-chain store	MinIO (or local disk), HashiCorp Vault	Ciphertext blobs only; key-encryption key custody. Private network segment, no client path.

3.2 Where truth lives

- **Chain = authorization truth.** Identity status, role balances, asset ownership, ACEs and proposals are written on chain and nowhere else.
- **Postgres = a rebuildable mirror.** The indexer follows the *finalized* block tag for authoritative state and *latest* for optimistic UI. If the mirror is ever corrupted it is dropped and replayed from block 0.
- **Read policy.** The mirror answers PUBLIC to CONFIDENTIAL requests (fast listings). SECRET and TOP_SECRET decisions call `effectivePermissions()` on chain. A background job compares mirror and chain on a sample every few minutes and raises an alert on any mismatch.
- **Write policy.** Every state-changing action (register, role mint or burn, asset mint or transfer, grant, revoke, approve) is a transaction signed in the actor's own wallet. The contract enforces the rules; the backend cannot bypass them.

3.3 Technology stack (single, consistent choice)

v2's PDF named Node/Express while its slide deck named Python/Flask; v3 named NestJS and FastAPI. v4 fixes one stack: Node.js for orchestration, Python only for the AI service.

Concern	Choice	Notes
Contracts	Solidity ^0.8.24, OpenZeppelin v5	Overflow checks by default; audited base contracts for ERC-721, ERC-1155, AccessControl, Pausable, ReentrancyGuard.
Contract tooling	Hardhat (deploy, unit tests); Foundry (invariant and fuzz tests)	Slither in CI; gas snapshots; Etherscan verification of every deployment.
Networks	Hardhat local → Ethereum Sepolia (demo) → Hyperledger Besu QBFT (production path)	Same bytecode on all three; QBFT is Besu's successor to IBFT 2.0.
Frontend	React + TypeScript, wagmi + viem, TanStack Query, Tailwind, Recharts	Wallet connection, typed contract calls, cached mirror queries, risk charts.
Backend	Node.js (NestJS or Express), viem, <code>siwe</code>	Auth, PEP, file service, indexer, decision log, anchoring job.
Database	PostgreSQL	Chain mirror plus operational tables (Section 12.2). Redis is optional for sessions and rate limits.
AI service	Python, FastAPI, scikit-learn (Isolation Forest)	Stateless scoring; optional SHAP and sentence-transformers.
Storage and keys	MinIO (S3 API) with local encrypted disk as demo fallback; HashiCorp Vault transit engine	Vault dev mode is acceptable for the demo only; the production path uses persistent storage and Shamir-split unseal.
Infrastructure	Docker Compose; GitHub Actions CI	Tests, invariants, Slither, lint on every push.

4. Identity layer

4.1 Components

- **ERC-1056 EthereumDIDRegistry**, deployed unmodified (or the canonical deployment on the target network). It issues the `did:ethr` identifier, tracks the identity's current *owner*, and supports time-limited delegate keys. This is the decentralized identifier the problem statement asks for.
- **ArgusIdentity**, an ERC-721 with the ERC-5192 locked pattern. It is the soulbound credential that ties a DID to one employee, carries the clearance level, and is visible in any wallet or block explorer.

```
contract ArgusIdentity is ERC721, IERC5192 {
    // soulbound identity credential
    enum Status { NONE, ACTIVE, SUSPENDED, REVOKED }
    struct Identity {
        address did; // ERC-1056 identity address -> did:ethr:<network>:<did>
        bytes32 empCommitment; // keccak256(employeeNo || salt): no raw employee ID on chain
        uint8 clearance; // 0 PUBLIC, 1 RESTRICTED, 2 CONFIDENTIAL, 3 SECRET, 4 TOP_SECRET
        Status status;
        uint64 issuedAt;
        uint64 validUntil; // tenure or contract expiry: the identity lapses automatically
    }
    mapping(uint256 => Identity) public identities; // identityId = SBT tokenId
    mapping(bytes32 => uint256) public byEmpCommitment; // one human -> one identity
    mapping(address => uint256) public byDid; // one DID -> one identity

    // Admin-only. The DID owner's EIP-712 signature is verified inside the contract.
    function registerIdentity(address did, bytes32 empCommitment, uint8 clearance,
        uint64 validUntil, uint256 deadline, bytes calldata sigByDidOwner)
        external returns (uint256 identityId);

    // Every identity-scoped action re-reads the LIVE ERC-1056 owner, never a cached address.
    modifier onlyController(uint256 id) {
        require(didRegistry.identityOwner(identities[id].did) == msg.sender, "NOT_CONTROLLER");
        _;
    }
}
```

4.2 Onboarding and cryptographic proof of DID control

Step	Actor	What happens	What is proven
1	Employee	Creates a wallet. The DID is <code>did:ethr:<network>:<address></code> ; it exists in ERC-1056 by default with no transaction.	Self-sovereign identifier, no central issuer.
2	HR / Admin	Verifies the employee out of band. Backend computes <code>empCommitment = keccak256(empNo salt)</code> with a random per-employee salt stored in Vault.	Raw employee number never touches the chain.
3	Employee	Signs an EIP-712 <code>Register(did, empCommitment, clearance, validUntil, nonce, deadline)</code> message.	Consent and control of the DID key.
4	Admin	Submits <code>registerIdentity()</code> . The contract recovers the signer and requires <code>signer == didRegistry.identityOwner(did)</code> , and checks that the caller holds the Admin role token.	On-chain cryptographic proof; a wrong signature reverts (demo step 2).
5	Contract	Reverts if the commitment or the DID is already registered. Mints the soulbound NFT to the DID address with status ACTIVE. Emits <code>IdentityRegistered</code> .	Uniqueness (Section 4.3).
6	Admin	Mints the base role token (User) to the identity. Clearance of SECRET or above needs a Security Officer co-approval (Section 5.7).	Least privilege from day one.

4.3 Three uniqueness guarantees, enforced by the contract

- `byEmpCommitment[c] != 0` reverts a second registration for the same employee: no duplicate humans.
- `byDid[did] != 0` reverts binding a DID that already belongs to an identity: no shared identities.
- `_update()` reverts on every transfer that is not a mint or a burn: the credential can never be sold, lent or reassigned.

These three rules directly answer the “unique user detection” weakness found in v2, and they live in the contract rather than in the backend.

4.4 Session authentication (SIWE) and step-up

```
POST /auth/nonce -> { nonce, expiresAt }
client builds an EIP-4361 message: domain, nonce, issuedAt, expirationTime, Resources: [ did:ethr:... ]
wallet signs it (personal_sign)
POST /auth/verify -> 1 check SIWE fields: domain, nonce, time window
                    2 signer = ecrecover(message, signature)
                    3 require didRegistry.identityOwner(did) == signer
                      (or validDelegate(did, "sigAuth", signer) for an approved device key)
                    4 identityId = byDid[did]; require status == ACTIVE and validUntil > now
                    5 issue JWT { identityId, roles, clearance, sid } valid 15 minutes,
                      plus a refresh token in an httpOnly cookie
```

- **Domain binding, nonce, issue time and expiry come for free** from EIP-4361, and a judge can look the standard up. This replaces v2's homemade challenge.
- **Authentication happens first and separately.** Only after step 4 does any role or ACE check run, which is exactly what demo step 2 shows.
- **Step-up re-authentication.** Reading a SECRET or TOP_SECRET asset, or any request whose risk score is 41–80, requires a fresh EIP-712 signature over `StepUp(action, resourceId, nonce, expiresAt)`. A hijacked session alone is then worth read access to low-classification data only.

4.5 Key rotation, suspension and revocation

- **Rotation.** `didRegistry.changeOwner(did, newKey)`. Grants are keyed on `identityId`, so nothing is re-minted or re-granted. The old key stops authenticating immediately because every check reads `identityOwner()` live. Device keys use `addDelegate()` with an expiry.
- **Why a leaked old key is harmless.** The DID address itself can still sign transactions with its original key, but ArgusChain authorizes by `identityOwner()`, not by that address. The asset NFTs held at the DID address also cannot be moved, because standard transfers always revert (Section 6.2).
- **Suspension.** A Security Officer can suspend an identity unilaterally (fast path: losing availability is safer than losing confidentiality). Un-suspending needs two approvals.
- **Revocation.** Setting status REVOKED (or burning the token) makes `effectivePermissions()` return 0 for that identity on every asset, atomically.

5. Access control: roles and per-asset permissions

5.1 Two tiers, one reason each

Tier	Contract	Question it answers	Why this primitive
A. Platform roles	RoleRegistry (ERC-1155)	What may this person do on the platform: onboard, mint, approve, audit?	Few, stable, per-person. A token is the right primitive, and mint/burn gives an audit event for free.
B. Asset permissions	AccessRegistry	What may this identity do to this particular asset, and until when?	A badge per (user, asset) cannot express read-but-not-download, cannot expire, cannot deny, and costs a mint per grant. A registry can.

The pitch line: “**We used tokens where tokens are the right primitive, and a purpose-built registry where they are not.**”

5.2 Roles

Token id	Role	Platform capabilities	Hard limits
1	Admin (PS)	Register identities, mint and burn role tokens, mint and allocate assets, propose grants, propose transfers.	No content bits without a second approver; never approves own proposal; cannot self-grant.
2	Manager (PS)	Hold assets, share within delegated permissions, approve CONFIDENTIAL proposals (not their own).	May delegate only bits they hold, within remaining delegation depth.
3	Auditor (PS)	Read metadata, ACLs, decisions and audit trail for every asset; see effective permissions of any identity.	Never holds P_READ, P_DOWNLOAD or P_WRITE (contract-enforced, invariant INV-8).

Token id	Role	Platform capabilities	Hard limits
4	User (PS)	Access assets according to ACEs granted to them.	No platform-level write capability.
5	Security Officer (extension)	Suspend identities, co-approve SECRET and TOP_SECRET proposals and clearance uplifts, view anomaly telemetry.	No content access. Removable without breaking PS compliance.

Roles 1–4 are exactly the problem statement. Role 5 exists because a defence-sector deployment needs someone other than the Admin to approve classified grants; it is an extension, not a replacement.

5.3 Separation-of-duty matrix

Capability	Admin	Manager	Auditor	User	Security Officer
Register identity, mint or burn a role token	✓	✗	✗	✗	✗
Mint and allocate an asset NFT	✓ (approvals from CONFIDENTIAL)	✗	✗	✗	✗
Propose a grant on an asset	✓	✓ own delegated subset	✗	✗	✗
Approve a CONFIDENTIAL grant	✓ not own	✓ not own	✗	✗	✓
Approve a SECRET or TOP_SECRET grant	✓ as second approver	✗	✗	✗	✓ at least one required
Read file content	✗ unless separately granted by another party	only if granted	✗ never	only if granted	✗ never
Read metadata, ACL and audit trail	✓	own assets	✓ all assets	own grants	✓ all assets
Suspend an identity	✗	✗	✗	✗	✓
View anomaly telemetry	✗	✗	✓	✗	✓

The two rows to defend

An administrator who can grant access cannot read the data, and an auditor who can see everything about a file cannot see the file. In v2 the Admin could do all of it. This single change is the strongest “we understand defence-sector requirements” signal in the design.

5.4 Permission bits

Bit	Name	Meaning
0	P_LIST	The asset appears in a listing (name and classification only).
1	P_READ_META	Size, hash, version, owner, ACL, history.
2	P_READ	View decrypted content in the application (rendered, no export).
3	P_DOWNLOAD	Export the raw file. Separate from P_READ so a Deputy Manager can view without exporting.
4	P_WRITE	Upload a new version.
5	P_SHARE	Delegate a subset of one's own permissions, bounded by delegation depth.
6	P_ADMIN	Edit the ACL, change classification (subject to the approval policy).
7	P_AUDIT	Full history and effective permissions of others (auditor bit).

5.5 Access control entries

```

uint32 constant P_LIST = 1<<0; P_READ_META = 1<<1; P_READ = 1<<2; P_DOWNLOAD = 1<<3;
uint32 constant P_WRITE = 1<<4; P_SHARE = 1<<5; P_ADMIN = 1<<6; P_AUDIT = 1<<7;
uint32 constant CONTENT_BITS = P_READ | P_DOWNLOAD | P_WRITE;

enum PrincipalType { NONE, IDENTITY, ROLE } // GROUP / UNIT arrive with Tier 3
function principalOf(PrincipalType t, uint256 id) pure returns (bytes32) {
    return keccak256(abi.encodePacked(uint8(t), id)); // one bytes32 for any subject
}

struct Ace {
    uint32 allowMask;
    uint32 denyMask;
    uint64 notBefore;
    uint64 expiresAt; // 0 = permanent; rejected for CONFIDENTIAL and above
    uint8 delegationDepth; // further re-shares allowed; 0 = terminal
    uint256 grantedBy; // identityId of the proposer
    bytes32 justificationHash; // keccak256 of the mandatory written reason
}

mapping(bytes32 => mapping(bytes32 => Ace)) internal aces; // resourceId => principal => ACE
// v4: resourceId = bytes32(tokenId). The same key can later be a folder-tree node id.

```

5.6 Resolving effective permissions

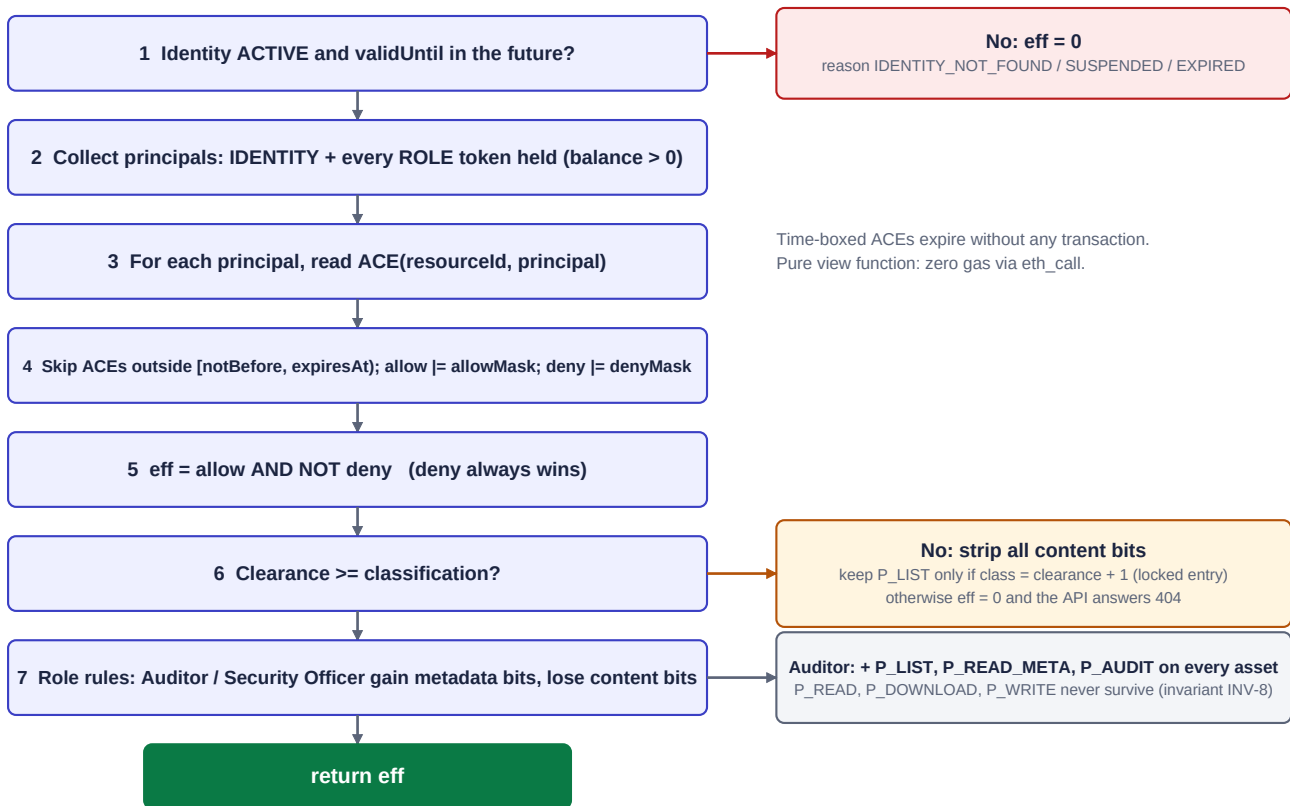


Figure 2. `effectivePermissions(resourceId, identityId)`: ACE resolution with deny override, then the mandatory access-control gate, then hard role rules.

```

function effectivePermissions(bytes32 resourceId, uint256 identityId) public view returns (uint32 eff) {
    Identity memory who = identity.get(identityId);
    if (who.status != Status.ACTIVE || block.timestamp >= who.validUntil) return 0;
    bytes32[] memory subs = _principals(identityId); // IDENTITY + every ROLE held
    uint32 allow; uint32 deny;
    for (uint256 i; i < subs.length; ++i) {
        Ace storage a = aces[resourceId][subs[i]];
        if (a.allowMask == 0 && a.denyMask == 0) continue;
        if (block.timestamp < a.notBefore) continue;
        if (a.expiresAt != 0 && block.timestamp >= a.expiresAt) continue;
        allow |= a.allowMask; deny |= a.denyMask; // deny is a global override
    }
    eff = allow & ~deny;
    eff = _mandatoryAccessControl(eff, who.clearance, assets.classificationOf(resourceId));
    eff = _roleRules(eff, identityId); // Auditor / Security Officer limits
}

```

- **Deny overrides allow, unconditionally.** Simpler than closest-ancestor-wins and strictly safer.

- **Grants expire on their own.** There is no cleanup transaction to forget. Default TTL is 90 days; caps by classification are in Section 5.7.
- **No self-grants.** `grant()` reverts when the target principal is the caller, and content bits granted to an Admin-role holder need a second approver.
- **Every ACE carries a justification hash** that is stored forever: a written, hash-committed reason for each grant is an audit requirement in any defence environment and costs one text field.
- **Visibility rule (resolves a v3 inconsistency).** v3's UI showed locked entries above clearance, while its API returned 404. v4 uses one rule: an asset one level above the caller's clearance appears as a greyed, locked entry (name and classification only) so users can file a justified access request; anything higher is invisible and the API answers 404.
- **Zero gas to read.** The function is a view. Gas is paid only when writing grants (on the order of 50–100k gas each).

5.7 Four-eyes grant workflow

```
enum ProposalType { GRANT, MINT_AND_ALLOCATE, TRANSFER, CLEARANCE_UPLIFT, ADMIN_ROLE_MINT }
struct Proposal {
    ProposalType kind; bytes32 resourceId; bytes32 principal;
    uint32 allowMask; uint32 denyMask; uint64 expiresAt;
    bytes32 justificationHash;
    uint256 proposer; uint8 approvals; uint64 readyAt;
    bool executed; bool rejected;
}
function propose(...) external returns (uint256 pid);
function approve(uint256 pid) external; // proposer != approver; clearance and role checks per table
function execute(uint256 pid) external; // anyone may call once approvals and readyAt are satisfied
```

Classification	Approvals (besides proposer)	Constraints	Timelock (T2)	Max TTL
PUBLIC / RESTRICTED	0	Proposer holds Admin, or Manager within delegation	none	365 days
CONFIDENTIAL	1	Approver is a different identity with role Admin, Manager or Security Officer and clearance ≥ CONFIDENTIAL	none	180 days
SECRET	2	Two distinct identities, different from proposer, at least one Security Officer, clearance ≥ SECRET	1 hour	90 days
TOP_SECRET	2	As SECRET, and every approver holds TOP_SECRET clearance	24 hours	30 days

The timelock is the anti-insider control: a compromised Admin cannot grant itself SECRET access and exfiltrate before the SOC sees a `ProposalCreated` alert. Emergency break-glass (Tier 2) needs a Security Officer plus an Admin, expires automatically after four hours, raises a P1 alert and is permanently flagged in the audit trail.

6. Asset layer: NFT ownership and integrity

6.1 AssetNFT

```
contract AssetNFT is ERC721 {
    struct Asset {
        bytes32 contentHash; // SHA-256 of the plaintext file (per version)
        uint8 classification; // 0 PUBLIC ... 4 TOP_SECRET
        uint32 version; // bumps on an approved re-upload
        uint256 ownerIdentity; // authoritative owner (identityId)
        uint64 mintedAt;
    }
    mapping(uint256 => Asset) public assets;
    mapping(bytes32 => uint256) public tokenOfHash; // the same content can never be minted twice

    // Caller's identity must hold the Admin role token (checked on chain).
    // Recipient: ACTIVE, clearance >= classification. ERC-721 holder = recipient's DID address.
    function mint(uint256 toIdentity, bytes32 contentHash, uint8 classification,
        bytes32 justificationHash) external returns (uint256 tokenId);

    function controlledTransfer(uint256 tokenId, uint256 toIdentity, bytes32 justificationHash) external;
    function transferFrom(address, address, uint256) public pure override {
        revert("USE_CONTROLLED_TRANSFER");
    }
}
```

- **Direct allocation.** The asset is minted to a registered identity, never to an anonymous address. `ownerIdentity` is authoritative; the ERC-721 holder address (the identity's DID address) is what wallets and block explorers show.
- **Admin-only creation.** The `mint()` modifier reads the Admin role balance of the caller's identity. For CONFIDENTIAL and above, minting is a `MINT_AND_ALLOCATE` proposal that needs the approvals in Section 5.7, because allocating a classified asset is effectively granting access to it.
- **Owner defaults.** On allocation the owner receives an ACE with `P_LIST`, `P_READ_META`, `P_READ`, `P_DOWNLOAD`, `P_WRITE` and `P_SHARE`. The Admin who minted it receives none of the content bits.
- **Honeypots are not flagged on chain.** Decoy assets are ordinary NFTs; the decoy marker lives only in a private table, so an observer of the chain cannot tell which asset is bait.

6.2 Controlled transfer

Standard `transferFrom` and `safeTransferFrom` revert for everyone, including the owner. `controlledTransfer()` is the only way to change ownership, and it requires a justification hash, the approvals for the asset's classification, and a recipient who is ACTIVE with sufficient clearance. It removes the previous owner's ACEs, emits `AssetTransferred`, and the standard `Transfer` event, so ownership history is simply the event log.

6.3 Integrity and tamper detection

Check point	How	Catches
Server, on every download	SHA-256 is computed incrementally while the file is decrypted and streamed; on mismatch with the on-chain <code>contentHash</code> the stream aborts and a tamper incident is logged.	Modified ciphertext or a swapped file in storage.
Browser, after download	WebCrypto recomputes SHA-256 of the received bytes and compares it with the hash read straight from the chain through a public RPC endpoint.	A compromised or lying backend. Independent of our server.
Background scan (T2)	Periodic sampled re-hash of stored objects.	Silent bit rot and offline tampering.

6.4 Versioning, leak detection and honeypots

- **Versioning.** `updateContent()` needs `P_WRITE` (and approvals for high classifications), bumps `version` and emits the old and new hash. Ownership history comes from `Transfer` events; content history from `AssetVersioned` events.
- **Semantic leak detection (T2).** A sentence-transformer embedding of the file is stored off chain. A reformatted or paraphrased leaked copy can then be matched to its original by cosine similarity, which a byte-level hash alone cannot do.

- **Honeypot assets (T2).** Decoy files with no business purpose. Any touch is a 100 percent-confidence signal: the caller receives a plausible fake success, and the SOC receives a P1 incident.

7. Storage and cryptography

7.1 Storage design

- **Bytes never touch the chain.** Only the SHA-256 content hash is on chain. Files live in MinIO (S3 API) on BEL-controlled infrastructure, with local encrypted disk as the offline demo fallback. IPFS is deliberately not used: “files stay on our secured infrastructure; the chain only verifies integrity” is the cleaner defence-sector story.
- **The application is the only door.** The bucket sits in a private network segment reachable only with the backend’s service credentials. There is no client-to-storage path, so the PEP check is not one door among several.
- **Layout.** Objects are stored as `/{{tokenId}}/{{version}}.enc`. Postgres holds tokenId, storage path, ciphertext hash, wrapped DEK and KEK version. The chain holds only the plaintext hash. Names and paths never go on chain.
- **Upload validation.** File-size limit and MIME allow-list are enforced before anything is hashed or written. An antivirus hook is listed as roadmap.

7.2 Envelope encryption with Vault

UPLOAD (size and MIME checks first)

```
1 server generates a random 256-bit DEK for this file version
2 ciphertext = AES-256-GCM(DEK, plaintext); contentHash = SHA-256(plaintext), computed while streaming
3 Vault transit wraps the DEK with the KEK -> wrappedDEK; the raw DEK is dropped from memory
4 ciphertext is written to MinIO at /{{tokenId}}/{{version}}.enc over the private network
5 Postgres stores tokenId, path, ciphertextHash, wrappedDEK, kekVersion (never a raw key)
6 the Admin signs mint(...) with contentHash in the wallet; only the hash reaches the chain
```

DOWNLOAD

```
1 session valid, PDP returned ALLOW with P_READ (or P_DOWNLOAD)
2 server asks Vault to unwrap the wrappedDEK using its own service identity
3 stream: read ciphertext -> AES-GCM decrypt -> SHA-256 incrementally -> send to the client
4 a mismatch with the on-chain contentHash aborts the stream and raises a tamper incident
5 the DEK is discarded; the browser re-verifies the hash on its own (Section 6.3)
```

The honest line for a judge

Compromising the application database alone does not yield a readable file: an attacker would also need live credentials to the Vault service. Rotating the KEK re-wraps every DEK with Vault’s rewrap operation without touching a single encrypted file, so rotation is a configuration operation, not an outage.

7.3 Server-side envelope now, client-side E2EE as the upgrade

Aspect	v4 build (from v2): server-side envelope	v4.2 upgrade (from v3): client-side E2EE
Who sees plaintext	The API server, briefly and only in memory; never persisted.	Only the browser of an authorized grantee.
Key handling	Per-file DEK wrapped by a Vault KEK; database holds only wrapped keys.	DEK wrapped per grantee to an x25519 public key (ECIES), plus an escrow wrap to a Vault-held key; Merkle root of wrapped keys committed on chain.
Cost of a grant	One ACE.	One ACE and a DEK re-wrap for the new grantee.
Revocation effect	Future access stops immediately; released plaintext cannot be recalled.	Re-key on revoke: new DEK, re-encrypt, re-wrap for remaining grantees only.
At scale	Backend proxies decrypted bytes.	Presigned single-object URLs for ciphertext; decryption happens in the browser.
Complexity	Low: Vault transit does the cryptography.	High. MetaMask removed <code>eth_decrypt</code> , so the x25519 key must be derived from a wallet signature.

The build takes the simpler path so the demo is reliable. The trade-off is stated openly: in the build, the API server is inside the trust boundary for plaintext. Escrow to a Vault-held key remains non-negotiable in the upgrade, because an employee who leaves must not take the only decryption path with them.

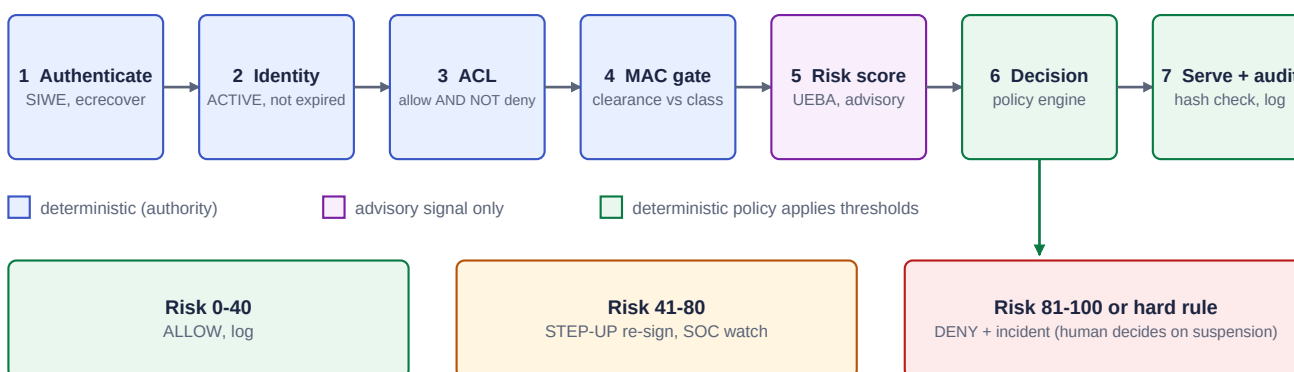
7.4 Revocation: state the limit before someone finds it

Revoking access stops all future server-mediated access. It cannot un-give plaintext that was already released. The three-part answer: **(1) attribute backwards**: every content release is a hash-chained, Merkle-anchored log line, so an investigation can prove who took what and when; **(2) re-key forward** (v4.2): a new DEK for later versions; **(3) watermark** (v4.2): rendered read-only views carry an invisible per-identity mark. Pitch wording: “Cryptographic revocation of already-released plaintext is impossible for any system, including ours. We make it detectable and attributable, and we make everything after the revocation inaccessible.”

7.5 Scaling notes

Component	What changes at scale
Storage	Local disk to MinIO distributed mode (erasure-coded, multi-node): a configuration change, not a redesign.
Hash verification	Already streaming: no second I/O pass. Add sampled background integrity scans for very high read volume.
Metadata database	Index by token id, owner identity and classification; partition by date or tier; read replicas for dashboard and audit queries.
On-chain writes	Grants and mints are the real bottleneck. A permissioned Besu network has no gas market; group or role principals grant to many people in one write; reads and denials are batched into Merkle roots.
AI service	Stateless: add instances behind a load balancer.

8. Authorization pipeline



Hard rules (honeypot touched, separation-of-duty violation, clearance probing, denial bursts) set the score to 100 and bypass the model.

Figure 3. Request lifecycle for reading an asset. Steps 1–4 and 6–7 are deterministic; step 5 is an advisory signal.

8.1 Enforcement split

- **State-changing actions are enforced inside the contracts.** Registering identities, minting roles and assets, granting, revoking, approving and transferring all revert on chain if the caller is not entitled. The backend cannot bypass them.
- **Content delivery is enforced by the PEP.** A chain cannot serve files, so the gateway asks the PDP on every request and streams bytes only on ALLOW. The PDP uses chain-derived state; its result is cached for at most 30 seconds and invalidated by the relevant chain events.
- **Crown jewels are not trusted to the mirror.** For SECRET and TOP_SECRET the PDP calls `effectivePermissions()` on chain. “We do not blindly trust our own database for the crown jewels” is a mature answer.
- **The model is never the authority.** The policy engine turns a number into an action using fixed thresholds. A false positive costs a step-up prompt, not a revoked engineer; suspension always needs a human.

8.2 Explainable denial: structured reason codes

Code	Meaning	HTTP	Surfaced to
IDENTITY_NOT_FOUND	Signer is not the owner of any registered DID	401	user
IDENTITY_SUSPENDED	Suspended by a Security Officer	423	user (no reason given)
IDENTITY_EXPIRED	Tenure <code>validUntil</code> has lapsed	403	user and HR queue
NO_ACE_ON_ASSET	No grant for this identity or its roles	403	user and suggested custodian to ask
EXPLICIT_DENY	A deny bit is set for this permission	403	user and admin
ACE_EXPIRED	The grant's TTL has passed	403	user, with one-click renewal request
PERMISSION_BIT_MISSING	Holds READ, requested DOWNLOAD	403	user, naming the missing bit
INSUFFICIENT_CLEARANCE	Clearance below classification	404 (locked entry if one level above)	user: no detail about the content
STEP_UP_REQUIRED	Risk 41–80, or SECRET+ read	401	user, with re-sign prompt
ANOMALY_SCORE_HIGH	Risk above 80	403 generic	user: generic; SOC: full explanation
HONEYPOT_TRIGGERED	A decoy asset was touched	200 fake success	SOC only (P1)
SOD_VIOLATION	Admin, Auditor or Security Officer tried to read content they may not	403	user and SOC
TAMPER_DETECTED	File hash differs from the on-chain hash	409	user: generic; SOC: incident

Two deliberate asymmetries: `INSUFFICIENT_CLEARANCE` never confirms what is inside (“you cannot see the `TOP_SECRET` budget” would reveal that one exists), and honeypots return a plausible fake success so the intruder does not learn they have been detected.

9. Audit trail

9.1 Every audit item in the problem statement is a native event

Problem statement audit item	On-chain event(s)	Emitted by
Identity creation	<code>IdentityRegistered</code> , ERC-721 <code>Transfer</code> (mint); <code>DIDOwnerChanged</code> on rotation	ArgusIdentity, ERC-1056
NFT creation	<code>AssetMinted</code> , ERC-721 <code>Transfer</code> (mint)	AssetNFT
Asset allocation	<code>AssetMinted.ownerIdentity</code> , <code>Transfer</code>	AssetNFT
Access-rights assignment (roles)	ERC-1155 <code>TransferSingle</code> (mint = grant, burn = revoke)	RoleRegistry
Access-rights assignment (assets)	<code>AceSet</code> , <code>AceRevoked</code> , <code>ProposalCreated</code> / <code>Approved</code> / <code>Executed</code>	AccessRegistry, GrantWorkflow
Ownership transfers	<code>AssetTransferred</code> , <code>Transfer</code>	AssetNFT
Permission updates	<code>AceSet</code> , <code>IdentityStatusChanged</code> , <code>ClearanceChanged</code> , <code>AssetVersioned</code>	AccessRegistry, ArgusIdentity, AssetNFT
Incidents and off-chain decisions	<code>AnchorPublished</code> (Merkle root of the decision log), <code>IncidentRecorded</code>	AuditAnchor

```

event IdentityRegistered(uint256 indexed identityId, address indexed did,
                        bytes32 empCommitment, uint8 clearance);
event IdentityStatusChanged(uint256 indexed identityId, uint8 status, uint256 actorIdentity);
event AssetMinted(uint256 indexed tokenId, uint256 indexed ownerIdentity,
                 bytes32 contentHash, uint8 classification);
event AssetTransferred(uint256 indexed tokenId, uint256 indexed fromIdentity,
                       uint256 indexed toIdentity, bytes32 justification);
event AssetVersioned(uint256 indexed tokenId, uint32 version, bytes32 oldHash, bytes32 newHash);
event AceSet(bytes32 indexed resourceId, bytes32 indexed principal, uint32 allow, uint32 deny,
             uint64 expiresAt, bytes32 justificationHash, uint256 grantedBy);
event AceRevoked(bytes32 indexed resourceId, bytes32 indexed principal, uint256 revokedBy);
event ProposalCreated(uint256 indexed pid, uint8 kind, uint256 indexed proposer);
event AnchorPublished(uint256 indexed batchId, bytes32 merkleRoot, uint256 fromId, uint256 toId);
event IncidentRecorded(uint256 indexed incidentId, bytes32 evidenceHash, uint8 severity);

```

9.2 Reads and denials: hash-chained log, Merkle-anchored

Chain events cover state changes. Reads and denials are too frequent to write individually, so each decision is appended to a hash-chained Postgres table (`row_hash = H(prev_hash || row)`) and a Merkle root is anchored on chain once per 1,000 lines or five minutes. Any log line is provable against the chain with a Merkle proof, and the Sepolia gas bill stays sane. Writing every event on chain would be architecturally unsound at BEL's volume.

- **Relayer least privilege.** The relayer that publishes anchors and incidents holds only an **ANCHORER** role: it can append roots, and it cannot mint, grant, read or approve. A stolen relayer key burns gas and nothing else.
- **Audit view.** A per-asset timeline of every grant, revoke, read and denial. Each on-chain row carries an Etherscan link; each off-chain row carries a Merkle proof check.
- **Privacy.** Only ids, hashes and masks are on chain: no names, paths or employee numbers. On the public Sepolia demo network the ACL graph is visible but opaque; the production Besu network is permissioned.

10. Behavioral threat detection (UEBA)

This is Layer 3 of the security model. The problem statement never mentions machine learning; this layer is the platform's chosen answer to the "Cybersecurity" half of the theme name. It is UEBA (User and Entity Behavior Analytics), an established security category alongside SIEM, not a generic "AI risk score".

Hard boundary, stated and built

This layer detects and explains. It never enforces. The deterministic policy engine is the only component with authority to allow or deny a request, and only humans can suspend an identity, burn a role token or approve a rule. That is the direct answer to "what if your model is wrong or gets fooled?": nothing catastrophic, because the model never held authority.

10.1 Feature families

Family	Features	Catches
Temporal	Hour-of-day and day-of-week z-score against the identity's own 30-day baseline; first activity after dormancy	Off-hours exfiltration
Volume	Requests and bytes in 1 minute / 1 hour / 24 hours, against own baseline and peer-group baseline	Bulk download
Breadth	Distinct assets per session, first-ever-access ratio, sequential-ID walking	Enumeration and scanning
Scope	Highest classification touched against 30-day maximum	Scope creep
Grant timing	Latency between a grant and the first access under it	Insider collusion, fresh-grant abuse
Denial	Denial rate, distinct reason codes, deny then retry on a different asset	Probing
Peer	Deviation from the centroid of same-role, same-unit colleagues	Removes "everyone works late" false positives

Peer-group baselining is the biggest accuracy upgrade over v2. A Deputy Manager pulling 40 files at 11 pm during a delivery crunch is anomalous against her own history but normal against her team's. That false positive is what makes analysts stop reading alerts.

10.2 Ensemble and thresholds

```

rule engine          hard signals (honeypot, SoD violation, denial burst, clearance probe) -> score 100
weighted z-score     T1 baseline score, directly explainable by construction
Isolation Forest     T2 point anomalies on the feature vector (no labelled attack data needed)
Markov surprisal     T3 sequence anomalies over action n-grams

risk = max(rules, IF_norm)          until the sequence model exists
risk = max(rules, 0.6*IF_norm + 0.4*seq_norm)  once it does

0-40    allow, log
41-80   allow after step-up, SOC watch
81-100  deny + P1 alert + recommend suspension (a human decides)

```

10.3 Explainability, cold start and honeypots

- **Explain-the-denial.** Never ship a bare score. Each flagged event returns its top three feature contributions in plain language (z-scores in T1, TreeSHAP for the forest in T2). Example: *Risk 87 (High): download volume 4.2σ above peer group; touched 7 distinct assets in 6 minutes against a baseline of 1.2; first-ever access to this asset, granted 4 minutes ago.* The last clause is a signal only a system with an on-chain grant timeline can produce; lead with it in the demo.
- **Cold start.** An Isolation Forest on an empty log is noise. Mitigations: a scripted synthetic “normal week” seed across about 30 personas, shadow mode (score and log, never enforce) for the first seven days of a new identity, and rule-engine-only decisions until an identity has 200 or more events. The build demonstrates the mechanism and the explanation pipeline, not production detection accuracy, and says so.
- **Honeypots.** A decoy asset touched by anyone is a hard rule: score 100, fake-success response, silent P1 to the SOC. No model is involved and there are no false positives.

10.4 Securing the AI layer itself

- **Signed scores, DID-delegate key custody.** The AI service signs its output with a key registered as an ERC-1056 delegate of an Oracle DID. The PDP and `AuditAnchor` check `validDelegate(oracleDid, "oracle", signer)` instead of a hard-coded address. Rotation and revocation are ordinary registry calls with no contract redeploy, and the same mechanism that authenticates humans now protects the AI service: one piece of infrastructure, two problems solved.
- **Prompt-injection defence (SOC Copilot, T3).** Any agent that reads filenames or log entries can be attacked through them. Three defences: read-only tools with no authority over access decisions, sanitised tool output before it enters the model context, and no irreversible action without human approval.

11. Smart-contract security and testing

11.1 Secure coding patterns

- Checks-effects-interactions ordering in every state-changing function; `ReentrancyGuard` wherever a function calls another contract.
- Explicit visibility everywhere; `msg.sender` only, never `tx.origin`; custom errors; an event for every state change.
- Solidity 0.8.x, so arithmetic overflow reverts by default. OpenZeppelin v5 base contracts throughout.
- Minting is Admin-gated on every token contract, so there is no open race to front-run.
- `Pausable` emergency stop for state-changing functions, triggered by a Security Officer; views and reads are never blocked.
- **Upgradeability.** The build deploys immutable contracts, which removes proxy-storage risk from the demo. The production path is UUPS proxies with storage gaps, owned by a 3-of-5 multisig behind a 48-hour timelock, never by a single account.
- **Bootstrap.** The deployer mints the first Admin role token, then renounces its own deployer role, so no privileged key outlives deployment.

11.2 Tooling and invariants

- **Slither** against every contract before the demo and in CI; document what it flagged and what was fixed. If it reports no high or medium findings, that is itself a one-sentence answer.
- **Hardhat unit tests** for every revert path; target at least 95 percent branch coverage on **AccessRegistry**. **Foundry** invariant tests for the properties below.
- **Roadmap only, not claimed:** Mythril, Echidna, formal verification and a third-party professional audit.

#	Invariant (must hold after any sequence of calls)
INV-1	No identity ever holds P_READ on an asset whose classification exceeds its clearance.
INV-2	A suspended, expired or revoked identity has effective permissions 0 on every asset.
INV-3	A deny bit set in any applicable ACE is never present in the effective mask.
INV-4	ArgusIdentity and RoleRegistry tokens never change owner except by mint or burn.
INV-5	An asset changes owner only through controlledTransfer , and no asset exists without an Admin-authorized mint.
INV-6	No ACE on a CONFIDENTIAL or higher asset exists without a matching executed proposal with the required distinct approvals.
INV-7	proposer != approver on every executed proposal.
INV-8	An identity holding the Auditor or Security Officer role never has P_READ, P_DOWNLOAD or P_WRITE.

Publish invariant results (runs per invariant, violations found) only after they have actually been run.

11.3 Threat model summary

Threat	Control	Layer
Stolen session token	15-minute JWT; fresh step-up signature for SECRET+ and every write	1
Forged identity or stale key	SIWE signer must equal live ERC-1056 owner; rotation is immediate	1
Reentrancy, access-control bugs	Patterns in 11.1, Slither in CI, invariants	2
Compromised Admin account	No content path; four-eyes and timelock for classified grants; SoD matrix	2
Insider abuse inside granted permissions	UEBA with peer baselines; honeypots; step-up on 41–80	3
Database compromise	Only wrapped DEKs stored; SECRET+ decisions read from chain; mirror is not authority	2 / storage
Stolen relay key	Anchor-only role: cannot mint, grant or read	2
AI service key leak	Revoke the ERC-1056 delegate; no redeploy	1 + 3
Model fooled or wrong	Advisory only; thresholds fixed in the policy engine; humans suspend	3
File tampering in storage	Server and browser hash checks against the chain	2 / storage
Released plaintext after revocation	Attribution log now; re-key and watermark in v4.2 (Section 7.4)	stated limit

12. Contract inventory and data model

12.1 Contracts

Contract	Standard	Key functions	Callable by	Tier
EthereumDIDRegistry	ERC-1056, unmodified	<code>identityOwner</code> , <code>changeOwner</code> , <code>addDelegate</code> , <code>validDelegate</code>	DID owner	T1
ArgusIdentity	ERC-721 + ERC-5192	<code>registerIdentity</code> , <code>suspend</code> , <code>setStatus</code> , <code>setClearance</code>	Admin; Security Officer; workflow	T1
RoleRegistry	ERC-1155, non-transferable	<code>grantRole</code> (mint), <code>revokeRole</code> (burn), <code>balanceOf</code>	Admin; Admin-role mint via workflow	T1
AssetNFT	ERC-721, controlled transfer	<code>mint</code> , <code>controlledTransfer</code> , <code>updateContent</code>	Admin; workflow; P_WRITE holder	T1
AccessRegistry	purpose-built	<code>setAce</code> , <code>revokeAce</code> , <code>effectivePermissions</code>	GrantWorkflow; P_ADMIN holders	T1
GrantWorkflow	purpose-built	<code>propose</code> , <code>approve</code> , <code>execute</code> , <code>reject</code> , <code>breakGlass</code>	per Section 5.7	T1 (timelock, break-glasses T2)
AuditAnchor	purpose-built	<code>anchor</code> , <code>recordIncident</code>	ANCHORER (relayer); oracle delegate	T2

12.2 Postgres tables

Chain-derived tables are written only by the indexer. Operational tables are written by the backend and never influence authorization.

Table	Key columns	Written by
identities	<code>identity_id</code> , <code>did</code> , <code>emp_commitment</code> , <code>clearance</code> , <code>status</code> , <code>valid_until</code> , <code>block_number</code>	indexer
role_balances	<code>identity_id</code> , <code>role_id</code> , <code>since_block</code>	indexer
assets	<code>token_id</code> , <code>content_hash</code> , <code>classification</code> , <code>owner_identity</code> , <code>version</code> , <code>block_number</code>	indexer
aces	<code>resource_id</code> , <code>principal</code> , <code>allow_mask</code> , <code>deny_mask</code> , <code>not_before</code> , <code>expires_at</code> , <code>granted_by</code> , <code>justification_hash</code>	indexer
proposals	<code>pid</code> , <code>kind</code> , <code>resource_id</code> , <code>principal</code> , <code>masks</code> , <code>proposer</code> , <code>approvals</code> , <code>ready_at</code> , <code>executed</code>	indexer
asset_labels	<code>token_id</code> , <code>name</code> , <code>folder_path</code> (off chain only)	backend
files	<code>token_id</code> , <code>version</code> , <code>storage_path</code> , <code>ciphertext_hash</code> , <code>wrapped_dek</code> , <code>kek_version</code>	backend
decisions	<code>id</code> , <code>ts</code> , <code>identity_id</code> , <code>resource_id</code> , <code>action</code> , <code>reason_code</code> , <code>risk_score</code> , <code>top_features</code> jsonb, <code>prev_hash</code> , <code>row_hash</code>	backend (hash-chained)
anchors	<code>batch_id</code> , <code>merkle_root</code> , <code>from_id</code> , <code>to_id</code> , <code>tx_hash</code> , <code>block_number</code>	backend and indexer
honeypots	<code>token_id</code> (private, never on chain)	backend
baselines, risk_events	feature vectors, peer cohorts, scores and reasons	AI service

12.3 API surface

Endpoint	Purpose	Guard
POST /auth/nonce, /auth/verify	SIWE login and session issue	public, rate-limited
POST /assets/upload	Validate, encrypt, store; returns contentHash for the mint transaction	Admin session
GET /assets	Listing, following the visibility rule of Section 5.6	session, PDP
GET /assets/:id	Metadata, hash, owner, version	P_READ_META
GET /assets/:id/content	Decrypted stream; full pipeline of Section 8	P_READ or P_DOWNLOAD, PDP, risk

Endpoint	Purpose	Guard
GET /assets/:id/acl	ACL editor data	P_ADMIN or P_AUDIT
GET /assets/:id/effective/:identityId	Effective Access trace	Auditor, or the identity itself
GET /audit, /audit/proof/:decisionId	Timeline and Merkle proof	Auditor, Security Officer, or own history
POST /internal/score	PDP to AI service	internal network only

13. Build plan: three tiers and a cut line

The scope is deliberately tiered. **Tier 1 is the complete, problem-statement-satisfying demo.** Tier 2 raises the security story. Tier 3 is presented as roadmap and never claimed as built.

Tier	Scope	Done means
T1 Must build	- ERC-1056 + ArgusIdentity (uniqueness, on-chain registration proof) - RoleRegistry (ERC-1155, non-transferable, bootstrap and renounce) - AssetNFT (Admin-only mint, unique hash, controlled transfer) - AccessRegistry (bits, deny, expiry) and effectivePermissions - GrantWorkflow core (proposer ≠ approver, approval counts by class) - SIWE login, indexer, Postgres mirror, PDP and PEP - File service: Vault envelope encryption, server and browser hash verification - UI: explorer, ACL editor, Effective Access view, audit timeline - Weighted z-score risk with plain-language reasons and reason codes - Hardhat tests on every revert path; Slither run and documented	Demo steps 1–7, 9 and 12 pass end to end, locally and on Sepolia.
T2 Should build	Timelocks and break-glass · AuditAnchor Merkle batching · Isolation Forest with peer baselines and TreeSHAP · honeypot assets · step-up EIP-712 for SECRET+ · DID-delegate oracle · semantic leak detection · Foundry invariants in CI · MiniIO in place of local disk	Demo steps 8, 10, 11 and 13 pass; invariants run in CI.
T3 Roadmap only	Namehash folder tree with inheritance · groups, org-unit principals and need-to-know compartments · client-side E2EE, re-keying and watermarking · Besu QBFT deployment · SOC Copilot · Markov sequence model · UUPS proxies with multisig and timelock · Mythril, Echidna, formal verification, third-party audit · ZK clearance proofs · proxy re-encryption · liveness or KYC hook at onboarding	Appears on the roadmap slide only.

13.1 Suggested order

- **Contracts first, in dependency order:** identity, roles, assets, access registry, workflow. Each is testable alone on a local Hardhat chain before anything else depends on it.
- **Then in parallel:** the backend (SIWE, indexer, PDP, PEP, file service) and the frontend (explorer, ACL editor, Effective Access). The AI service can start as soon as the indexer produces access events.
- **Cut from the bottom without hesitation.** A flawless demo of Tier 1 beats a buggy demo of everything, and Tier 2 and 3 items presented as a credible roadmap read as maturity rather than gaps.

13.2 v4.1 extension path

The design leaves the door open without paying for it now. **AccessRegistry** is keyed by **resourceId**, which is **bytes32(tokenId)** today. A namehash folder tree (**nodeId = keccak256(parent ◻ labelHash)**, classification never decreasing down the tree, NTFS-style inheritance break) can replace that key later without changing the ACE structure, the workflow or the audit events.

14. Demo script

Each step is chosen so that a claim in Section 2 becomes visible instead of merely stated. Failure steps (2, 3, 7, 9, 12) matter as much as success steps.

#	Action	What the judge sees	Proves
1	Admin onboards a new employee.	DID created; employee signs the registration; a soulbound identity NFT appears in the wallet and on the block explorer.	DID, identity credential
2	Sign in (or register) with a signature that does not match the DID owner.	Rejected at authentication, before any role or ACL check runs.	Cryptographic authentication is separate from authorization
3	Try to onboard the same employee, or the same DID, again.	Contract reverts.	Unique identity
4	A non-admin tries to mint an asset; then the Admin mints one. Try to mint the same file again.	First reverts. Second shows the hash and an NFT under the employee's identity. Third reverts as a duplicate.	Admin-only mint, allocation, no duplication
5	Admin mints the Manager role token; try to transfer it peer to peer.	Balance change visible on screen; transfer reverts.	RBAC by mint/burn
6	Admin grants a User a 14-day read-only ACE; open the Effective Access view; request DOWNLOAD.	Line-by-line trace with transaction links; DOWNLOAD denied with PERMISSION_BIT_MISSING.	Per-asset permissions, explainable denial
7	Admin tries to read the content; the Auditor opens the same asset.	Admin: SOD_VIOLATION. Auditor: full metadata, ACL and audit trail, no content.	Separation of duty
8	Propose a SECRET grant, then try to approve it from the proposer's account.	Self-approval reverts; a Security Officer and a second approver are required; timelock counts down.	Four-eyes control
9	Rotate the employee's key with <code>changeOwner</code> .	Old key rejected at login; new key works; every grant is intact.	Identity-keyed grants
10	Simulate compromise: new device, 2 am, rapid asset enumeration.	Risk spikes, plain-language explanation shown, step-up then Deny plus incident even though the ACL would allow it.	Layer 3, AI advises, policy decides
11	Touch a honeypot asset.	Fake success to the caller; instant P1 incident on the SOC dashboard.	Deception technology
12	Flip one byte of a stored file in MinIO and download.	Server aborts the stream; the browser independently rejects it against the on-chain hash.	Tamper evidence
13	Open the audit view; verify one off-chain row.	Etherscan links for state changes; a Merkle proof verified against an anchored root.	Immutable audit trail

15. Anticipated judge questions

Question	Answer
"A role-token balance check is not cryptographic authentication. Where is the proof?"	Authentication is separate and comes first: registration verifies the DID owner's EIP-712 signature inside the contract, and every session verifies a SIWE signature against the live ERC-1056 owner. The role or ACL check runs only afterwards. Demo step 2 shows a wrong signature dying before any authorization runs.
"Why not make everything a token?"	Tokens are used where they are the right primitive: identity, roles, assets. A badge per (user, file) cannot express read-but-not-download, cannot expire and cannot deny, so per-asset permissions live in a purpose-built registry.
"Isn't the AI layer outside the problem statement?"	Yes, explicitly. It answers the Cybersecurity half of the theme name. The blockchain core satisfies the problem statement on its own (Section 2), and the AI layer has no authority to enforce anything.
"What if your model is wrong or gets fooled?"	Nothing catastrophic. It only produces a number and a reason; fixed thresholds in the policy engine turn that into a step-up prompt or a deny, and only humans can suspend an identity or change a rule.
"An admin can grant anything. What stops a rogue admin?"	The Admin has no path to content. Grants on CONFIDENTIAL and above need a different approver; SECRET and above need a Security Officer, a timelock and a TTL cap; every grant carries a permanent written justification hash.
"Can you revoke a file someone already downloaded?"	No system can. We stop everything after the revocation, attribute everything before it through the anchored release log, and (v4.2) re-key and watermark.
"Your ACL graph is public on Sepolia."	Only opaque ids, hashes and masks are on chain: no names, paths or employee numbers. Sepolia is the demo network; the same bytecode deploys unmodified to a permissioned Besu network.
"Where does the encryption key live?"	A per-file key encrypts the file; a master key inside Vault wraps it; only the wrapped key sits in Postgres. A database dump alone yields nothing readable.
"What if the AI signing key or the relayer key leaks?"	The AI key is an ERC-1056 delegate and is revoked with one registry call, no redeploy. The relayer holds an anchor-only role and cannot mint, grant or read.
"How is this different from a database with a permissions table?"	Every line of the Effective Access view traces to an on-chain event with a transaction hash, permissions are enforced inside contracts rather than by trust in an operator, and the audit trail cannot be edited by the people it audits.
"Your anomaly model is trained on synthetic data."	Correct, and stated. The build demonstrates the mechanism and the explanation pipeline, runs in shadow mode for new identities, and falls back to hard rules below 200 events. It is not claimed as production-grade detection accuracy.

16. Known risks and honest scope

Risk	Reality	Our answer
Public ACL graph on Sepolia	Anyone can see which opaque principal holds which mask on which asset id.	No names, paths or employee ids on chain; show an Etherscan tab of opaque bytes32 values; production is a permissioned Besu network.
Gas cost per grant	Roughly 50–100k gas per ACE; a 500-person onboarding is expensive.	Role principals grant to a whole role in one write; batching; permissioned chain has no gas market.
Sepolia is not production	Testnet, faucet-dependent, probabilistic finality.	Identical bytecode on Besu QBFT. The indexer reads the <i>finalized</i> block tag for authoritative state.
API server sees plaintext (build)	Server-side envelope encryption puts the server inside the trust boundary for plaintext, in memory only.	Stated openly; Vault-only key custody; client-side E2EE is the documented upgrade (Section 7.3).
Vault as a single point	Vault unwraps every DEK.	Shamir-split unseal, audit logging of every unwrap, SOC alert on unusual unwrap volume. Dev mode is for the demo only.
Relayer key compromise	A stolen key could publish bogus anchors.	Anchor-only role: cannot mint, grant, approve or read; anchors are verifiable against the hash-chained log.
ML cold start	A forest on thin data is noise.	Synthetic seed, shadow mode, rule-only fallback below 200 events.
Scope overrun	The full design is larger than a hackathon.	Three tiers and a cut line; only Tier 1 and Tier 2 are ever claimed.
Revocation is not forgetting	Released plaintext cannot be recalled.	Attribute backwards, re-key forward (v4.2), watermark (v4.2).
ERC-1056 deployment differs per network	Registry addresses and DID network prefixes differ between chains.	Deploy or reference the canonical registry per network through one configuration file and test the resolver on each.

Appendix A. Traceability: v2 and v3 features in v4

Source	Feature	v4 disposition
v2	did:ethr via ERC-1056 + signature check	Kept (Sections 4.1–4.4); registration proof moved on chain.
v2	Soulbound identity NFT (ERC-5192 pattern)	Kept; now also holds clearance, validity and uniqueness commitments.
v2	ERC-1155 role tokens (Admin, Manager, Auditor, User)	Kept; one extension role added (Security Officer).
v2	ERC-721 asset NFTs with SHA-256 hash, Admin-gated mint	Kept; unique-hash rule and controlled transfer added.
v2	Envelope encryption with Vault transit	Kept as the build's encryption model (Section 7.2).
v2	Three-layer security model	Kept as the security framing (Sections 4, 10, 11).
v2	DID-delegate oracle key custody	Kept (Section 10.4); scores are signed by an ERC-1056 delegate.
v2	Honeypots, explain-the-denial, semantic leak detection	Kept as Tier 2 (Sections 6.4, 10).
v2	Slither before demo; roadmap-only audit	Kept; Foundry invariants added.
v2	Admin can do everything	Replaced by separation of duty (Section 5.3).
v3	Unified principal (identity, designation, group, unit)	Modified: IDENTITY and ROLE in build; GROUP and UNIT are Tier 3.
v3	Grants keyed on identityId, not msg.sender	Kept; identityId ties to the DID whose owner is read live.
v3	AccessRegistry: bitmasks, deny override, expiry, delegation	Kept, resource keyed by tokenId (Sections 5.4–5.6).
v3	Four-eyes workflow, timelock, break-glass	Kept; simplified approval rules; timelock and break-glass are Tier 2.
v3	Separation-of-duty matrix	Kept (Section 5.3).
v3	SIWE login and step-up signatures	Kept (Section 4.4).
v3	Chain as truth, Postgres as rebuildable mirror; eth_call for SECRET+	Kept (Section 3.2).
v3	Merkle-anchored, hash-chained audit log	Kept (Section 9.2).
v3	Peer-group baselining, reason codes, Effective Access view	Kept (Sections 8, 10, 14).
v3	Foundry invariants	Kept, adapted to the v4 contracts (Section 11.2).
v3	Namehash resource tree, inheritance break	Deferred to Tier 3; extension path in Section 13.2.
v3	Client-side E2EE, ECIES key bundles, re-keying	Deferred to v4.2 (Section 7.3).
v3	Designation ladder and BEL grades	Deferred ; clearance and roles cover the build.
v3	UUPS proxies on every contract	Deferred to the production path (Section 11.1).
v3	Files as registry nodes with no NFT; no DID	Dropped : conflicts with the problem statement.
v3	Fixed 12-confirmation indexer	Modified: read the <i>finalized</i> block tag.
v3	Locked-entry UI versus 404 API (inconsistent)	Resolved with one visibility rule (Section 5.6).

Appendix B. Changes needed in the SIH presentation

The submitted deck describes v2. Update these points so the deck and the build match.

Slide	Change
2 Proposed solution	Add a bullet for per-asset permissions (bits, deny, expiry, justification) and separation of duty: Admin grants but cannot read; Auditor sees everything but the file; four-eyes on classified grants.
3 Technical approach	Replace “Backend: Python / Flask” with Node.js orchestration plus a Python FastAPI AI service. Add AccessRegistry and GrantWorkflow to the blockchain layer. The existing DEK / KEK / Vault description already matches the v4 encryption model.
3 Users and clients	Add the Security Officer role as an extension to the four PS roles.
4 Feasibility	Add a challenge and mitigation pair: scope tiers with a cut line; server-side plaintext exposure with the client-side E2EE upgrade.
5 Impact	Add “no single account can both grant and read” to the security benefits.
6 References	Add EIP-4361 (SIWE), EIP-712, and the OpenZeppelin AccessControl and Pausable documentation; keep the existing ERC and Isolation Forest references.

End of document. ArgusChain v4 supersedes ArgusChain v2 and v3 for build and presentation purposes.